

EXAMEN 2do Parcial

① Es el registro especial del procesador donde se guarda el resultado de las operaciones aritméticas y lógicas, y el estado general del procesador.

- Tiene banderas de estado (Z: resultado cero, V desbordamiento, etc)
- El ALU lo modifica automáticamente
- Usado para saltos condicionales

② \$Zero (0) -- constante 0

\$at (1) -- para ensamblador

\$v0 (2) -- evaluación de expresión y resultado de función

\$v1 (3) -- evaluación de expresión y resultado de función

\$a0 - \$a3 (4-7) -- pasan los primeros 4 argumentos a rutinas y los restantes a pila

\$t0 - \$t9 (8-25) -- preservan su valor ante llamadas a procesamiento usados para compilar el programa en instrucciones MIPS

\$s0 - \$s7 (16-23) -- preservan el valor ante llamadas a procedimiento usados para variables globales en C

\$gp (28) -- puntero global a la mitad del bloque de 64kb de memoria del segmento de datos estáticos

③ Límite superior: 2^{32}

$$2^{32} = 4,294,967,296 \text{ bytes} = 4 \text{ GB}$$

Hexadecimal: 0x100000000

\$sp (29) -- puntero de pila a la última localidad de la pila

\$fp (30) -- apuntador de marco apunta a la primera palabra del marco de un procedimiento

\$ra (31) -- instrucción jal escribe el registro \$ra, da la dirección de retorno de la llamada a un procedimiento.

④

Jal 0x00400024 [Main]

-- llama al main

addi \$16, \$0, 40

-- carga el valor de 40 al registro \$50 con una suma inmediata

sw \$16, 8(\$5)

-- guarda el valor de \$50 en la direccion \$a1 + 8

lw \$7, 8(\$5)

-- carga el valor que esta en (\$a1 + 8) en \$a3 (el valor 40)

addi \$4, \$7, 50

-- suma el valor 50 del registro \$a3 y lo guarda en \$a0

addi \$2, \$0, 1

-- le asigna el codigo de imprimir entero a \$v0 (1)

syscall

-- llama a la función e imprime 40 en consola

addi \$2, \$0, 10

-- le asigna el codigo de salir del programa a \$v0 (10)

syscall

-- llama a la función y termina el programa

- ⑤
- SISD: Single Instruction single data stream
Un monoprocesador convencional tiene un flujo de instrucciones y un flujo de datos
 - MIMD: Multiple instruction Multiple data
En un multiprocesador hay varios flujos de instrucciones y varios flujos de datos.
 - SIMD: Single Instruction Multiple Data
Podría sumar 64 números enviando 64 flujos de datos a 64 ALU's para hacer 64 sumas en un único ciclo.
 - MISD: no tiene aplicaciones actualmente, aunque han surgido propuestas para diseñar procesadores MISD para crear redundancia de datos, pero se ha argumentado que estos serían más robustos en cuanto a tolerancia de errores.